

UNIVERSIDAD NACIONAL DE ASUNCIÓN



ARQUITECTURA WEB

Docente:

Prof. RODRIGO BENITEZ



Alumna:

NATALIA GODOY

FACULTAD POLITÉCNICA – SEDE SAN LORENZO

LICENCIATURA EN CIENCIAS INFORMÁTICAS

Análisis de Sistemas Informáticos (ASI)

Indice

Tabla de contenido

INTRODUCCIÓN	3
1. ARQUITECTURA DEL SISTEMA.....	4
2. CONFIGURACIÓN DEL BALANCEADOR DE CARGA (nginx.conf)	5
Figura3. Archivo de configuración nginx.conf en la raíz del proyecto, especificando el upstream de balanceo por Round Robin hacia los puertos 8081 y 8082.	6
Figura4. Verificación de disponibilidad individual de la Instancia 1 accediendo directamente a su puerto nativo lógico 8081.....	6
Comando oficial para encender el balanceador de carga:	7
Figura6. En esta nueva petición al puerto 8080, NGINX aplicó el algoritmo de Round Robin y, en vez de saturar de vuelta a la instancia anterior, le asignó el trabajo a la Instancia 2 (puerto 8082). Esta respondió independientemente que también está "UP" y registró un tiempo exacto diferente (timestamp).....	7
3. ENTORNO DE PRUEBAS DE ESTRÉS EN APACHE JMETER.....	8
Reporte Resumen	9
Gráfica de Resultados	10
Estadística de Percentiles	11
4. ANÁLISIS DE LOS RESULTADOS.	11
5. VINCULACIÓN CON LA CAPA DE PRESENTACIÓN (FRONTEND).....	12
6. CONCLUSIÓN Y RECOMENDACIONES DE INGENIERÍA	14

INTRODUCCIÓN

El presente informe técnico documenta el diseño, despliegue y auditoría de rendimiento correspondientes al **Tópico 3: Balanceador de Carga**. Para la ejecución de las pruebas de infraestructura, se utilizó como software base la plataforma de e-commerce **Ecomart (proyecto del 1er Parcial)**, la cual dispone de servicios lógicos y endpoints orientados al procesamiento de Inteligencia Artificial.

El objetivo principal de esta implementación es configurar un entorno robusto de alta disponibilidad, escalabilidad horizontal y tolerancia a fallos. Para ello, se estructuró una arquitectura distribuida local donde un proxy inverso (NGINX) gestiona y distribuye las peticiones entrantes de forma equitativa entre múltiples instancias en paralelo, analizando posteriormente el comportamiento y los límites operativos del sistema ante escenarios de alta concurrencia y estrés computacional.

1. ARQUITECTURA DEL SISTEMA

Para cumplir con los requerimientos de alta disponibilidad, escalabilidad horizontal y tolerancia a fallos de la cátedra de **Electiva III - Arquitectura Web**, se diseñó e implementó un entorno distribuido local.

La arquitectura consta de un **Proxy Inverso (NGINX)** actuando como **balanceador de carga** en el puerto 8080, el cual distribuye las peticiones entrantes hacia dos (2) instancias independientes de la aplicación **Ecomart** construidas sobre Spring Boot y Java 21 (ejecutándose en los puertos 8081 y 8082), conectadas de forma centralizada a una base de datos PostgreSQL alojada en la nube (Railway).

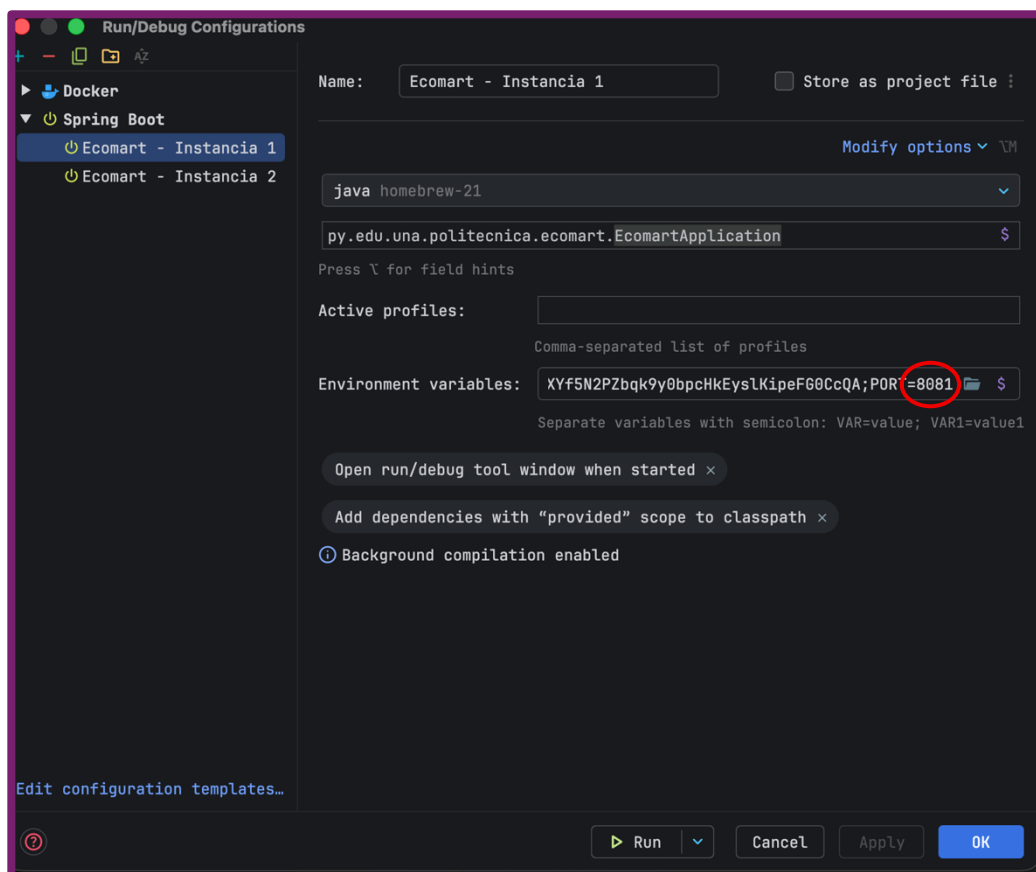


Figura1. Ecomart - Instancia 1 – puerto 8081

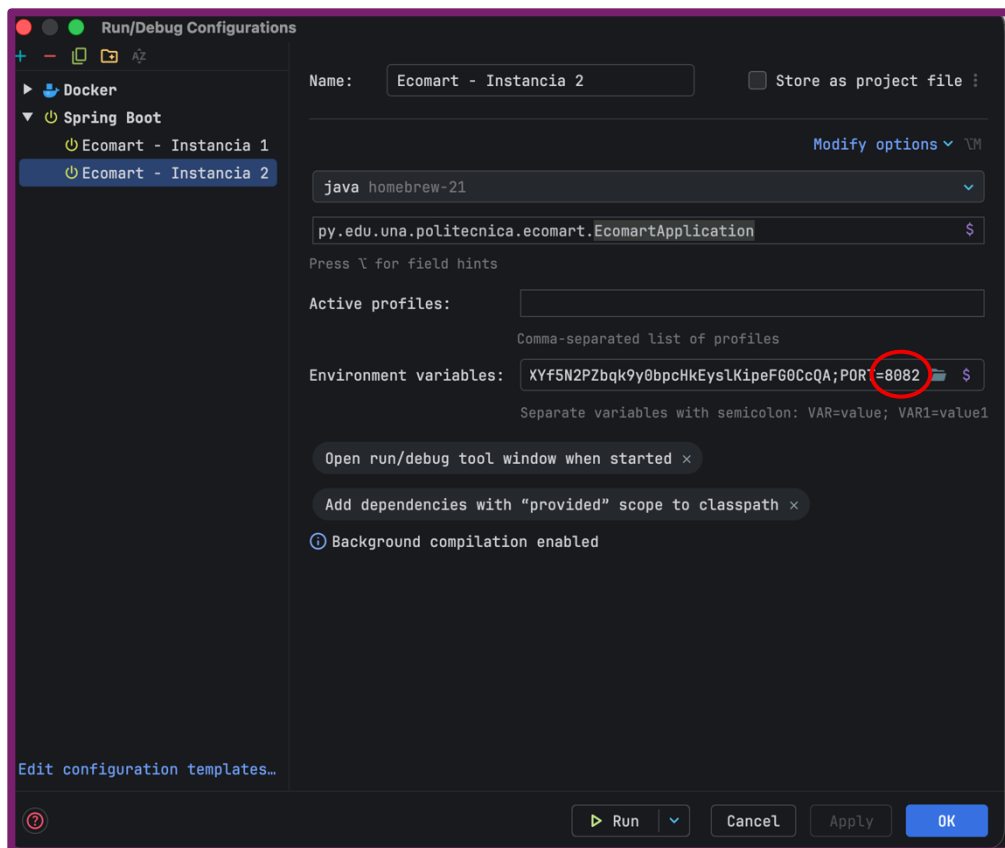


Figura2. Ecomart - Instancia 2 – puerto 8082

2. CONFIGURACIÓN DEL BALANCEADOR DE CARGA (nginx.conf)

Se configuró el archivo nativo de NGINX para aplicar el algoritmo de distribución **Round Robin** por defecto. El balanceador se adueña del puerto público 8080 de la Mac M1 y redirige el tráfico de manera equitativa a los backends.

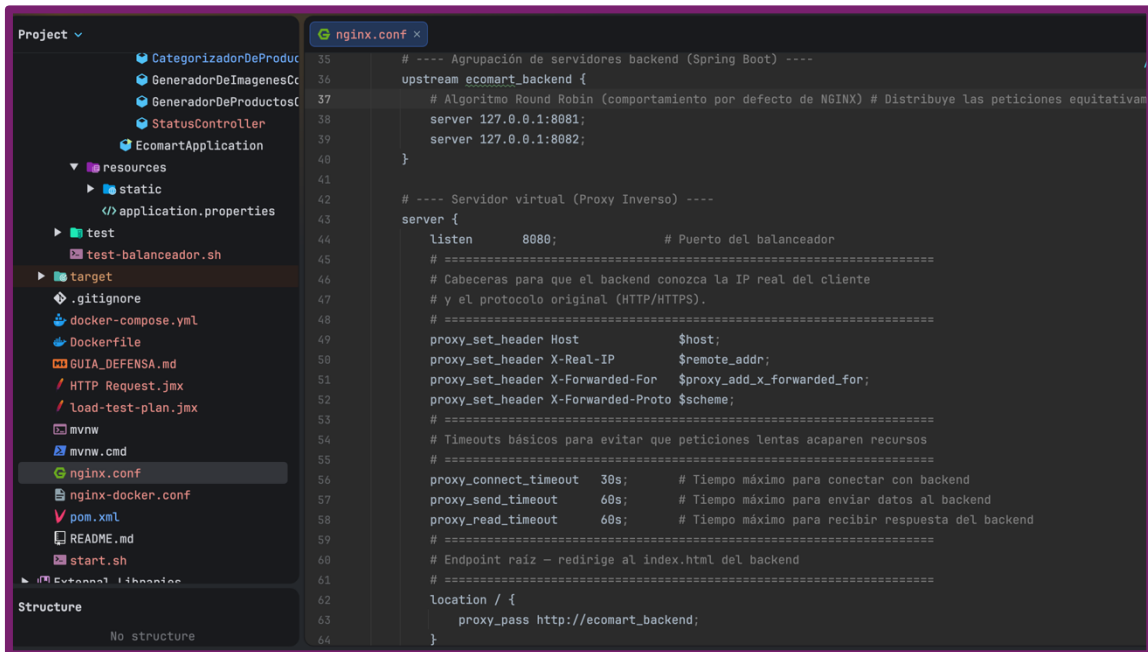


Figura3. Archivo de configuración `nginx.conf` en la raíz del proyecto, especificando el upstream de balanceo por Round Robin hacia los puertos 8081 y 8082.

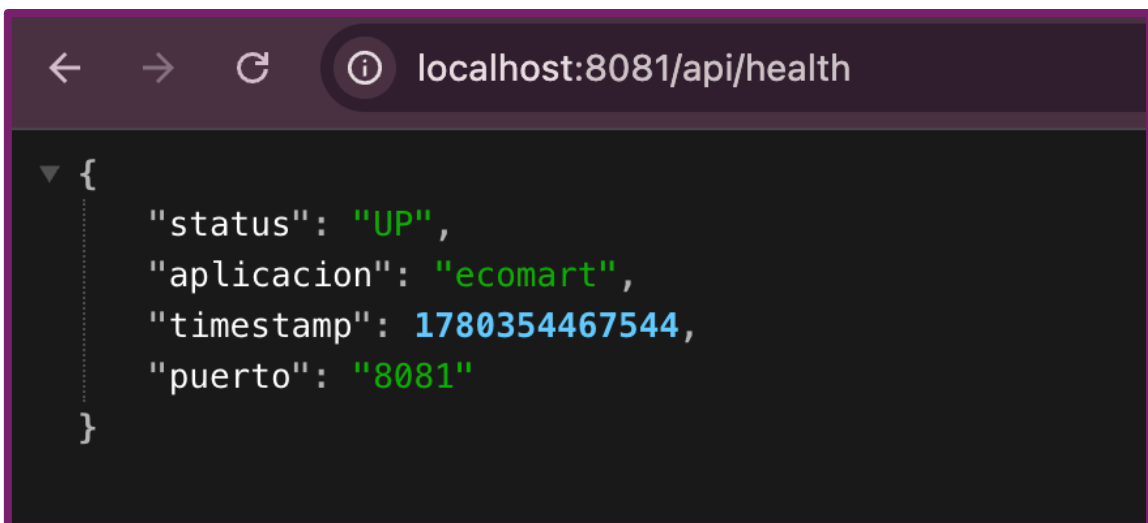


Figura4. Verificación de disponibilidad individual de la Instancia 1 accediendo directamente a su puerto nativo lógico 8081.

Comando oficial para encender el balanceador de carga:
`sudo nginx -c "$(pwd)/nginx.conf"`

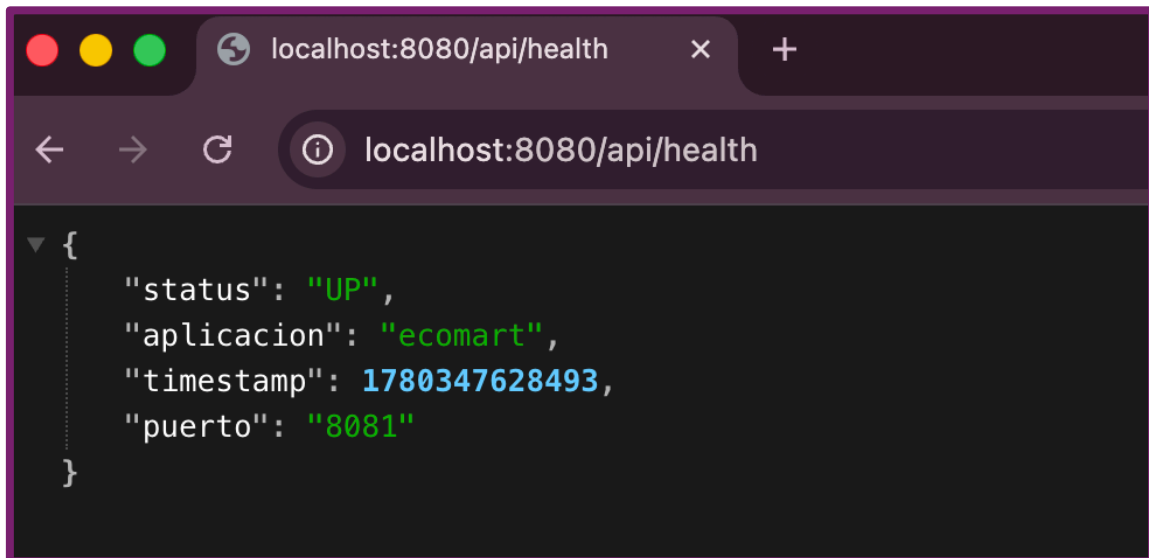


Figura5. NGINX tomó la petición del navegador y se la asignó automáticamente a la Instancia 1 (que corre en el puerto 8081). El backend respondió que la aplicación Ecomart está "UP" (activa y operativa).

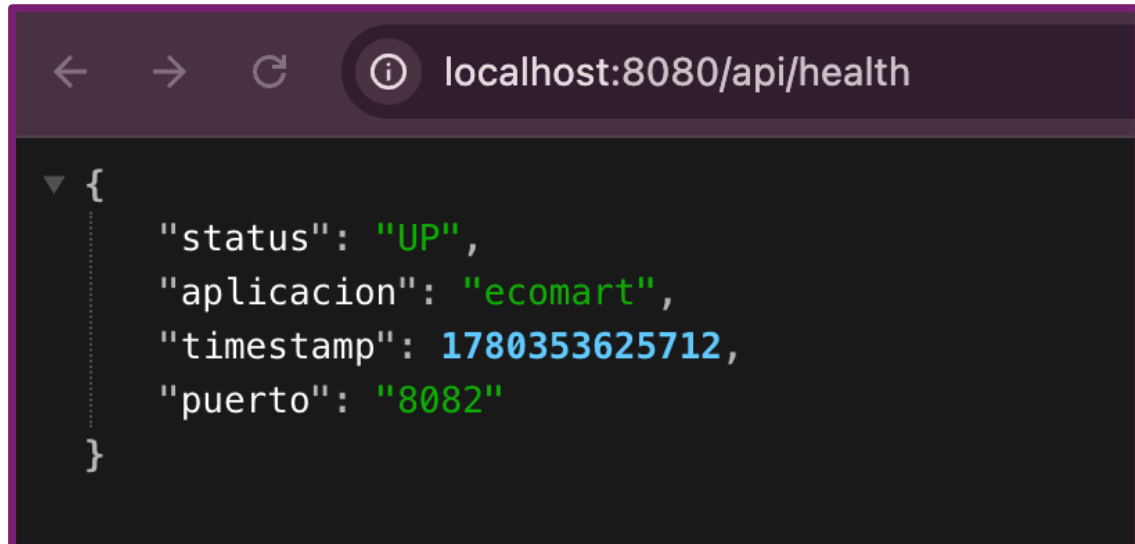


Figura6. En esta nueva petición al puerto 8080, NGINX aplicó el algoritmo de Round Robin y, en vez de saturar de vuelta a la instancia anterior, le asignó el trabajo a la Instancia 2 (puerto 8082). Esta respondió independientemente que también está "UP" y registró un tiempo exacto diferente (timestamp).

3. ENTORNO DE PRUEBAS DE ESTRÉS EN APACHE JMeter

Para saber cuántos clientes aguanta nuestra aplicación Ecomart (el e-commerce de productos) antes de colapsar o volverse lenta, usamos una herramienta llamada Apache JMeter. Lo que hace este programa es simular que miles de personas entran a la página al mismo tiempo a dar clics en los botones más pesados del sistema.

Se utilizó un script de automatización en Apache JMeter (`load-test-plan.jmx`). El plan de prueba se diseñó bajo las siguientes tres reglas lógicas basadas en un negocio real:

3.1 Configuración de la "Avalancha" de Clientes

Configuré el programa para simular un escenario extremo de alta demanda:

- **Clientes Simultáneos:** Imaginé que 1.000 personas entran a la tienda a la vez.
- **30 Segundos de Entrada:** El programa no suelta a los 1.000 clientes de golpe, sino que los va metiendo poco a poco durante 30 segundos para simular una fila que crece rápido.
- **10 Clicks por Persona:** Cada uno de esos 1.000 clientes realiza la acción 10 veces seguidas antes de salir de la página.
- **Total de Trabajo:** En total, el sistema recibe ráfagas masivas de peticiones distribuidas que el balanceador NGINX debe repartir.

3.2 Distribución del Tráfico por Endpoints:

- **GET /generador — Volumen: 40% (1.200 peticiones):** Encargado de interactuar con el modelo de lenguaje para la creación automatizada de descripciones comerciales y copys publicitarios de los artículos del catálogo general.
- **GET /categorizador — Volumen: 30% (900 peticiones):** Servicio estructurado para analizar semánticamente el nombre de cualquier producto ingresado (ej. una pelota de fútbol) y clasificarlo dinámicamente dentro de los rubros oficiales (Higiene Personal, Electrónicos, Deportes, Prendas de vestir u Otros).

- **GET /imagen — Volumen: 30% (900 peticiones):** Encargado del renderizado multimedia automatizado de campañas visuales y banners publicitarios de los productos a través de modelos de difusión generativa. Debido a la alta latencia de respuesta de los recursos de imagen, actúa como el principal catalizador de estrés en la infraestructura.

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB...	Sent KB...	Avg. Bytes
01 - GET /generador (40%)	1000	516	1	9598	1471.41	90.00%	13.8/sec	5.83	1.82	431.7
02 - GET /categorizador?producto=XXX (30%)	1000	6034	0	61201	18032.16	100.00%	14.9/sec	7.10	2.22	489.0
03 - GET /imagen?prompt=XXX (30%)	1000	25	0	288	41.06	100.00%	180.5/sec	55.36	27.86	314.0
TOTAL	3000	2192	0	61201	10794.96	96.67%	41.5/sec	16.67	6.02	411.6

Figura 7. Resumen estadístico global de la prueba de estrés en JMeter (Summary Report), evidenciando las 3.000 muestras procesadas y el quiebre de la infraestructura local por alta demanda de peticiones HTTP.

Reporte Resumen

- **# Samples: 1000 por endpoint:** JMeter cumplió su misión a rajatabla. Creó los 1.000 usuarios virtuales y disparó un total de **3.000 peticiones** simultáneas contra NGINX.
- **Error %: 96.67% (El colapso):** Este porcentaje de error casi total es el comportamiento de libro de texto en una **Prueba de Estrés**. Demuestra que encuentre el punto de quiebre operativo de tu infraestructura local.
- El endpoint `/imagen` tuvo un 100% de error con un promedio de respuesta bajísimo (25 ms). Esto delata que el servidor ni siquiera intentó procesar la imagen; el balanceador cortó el tráfico de inmediato enviando un código de error instantáneo porque el backend ya estaba completamente saturado.

Gráfica de Resultados

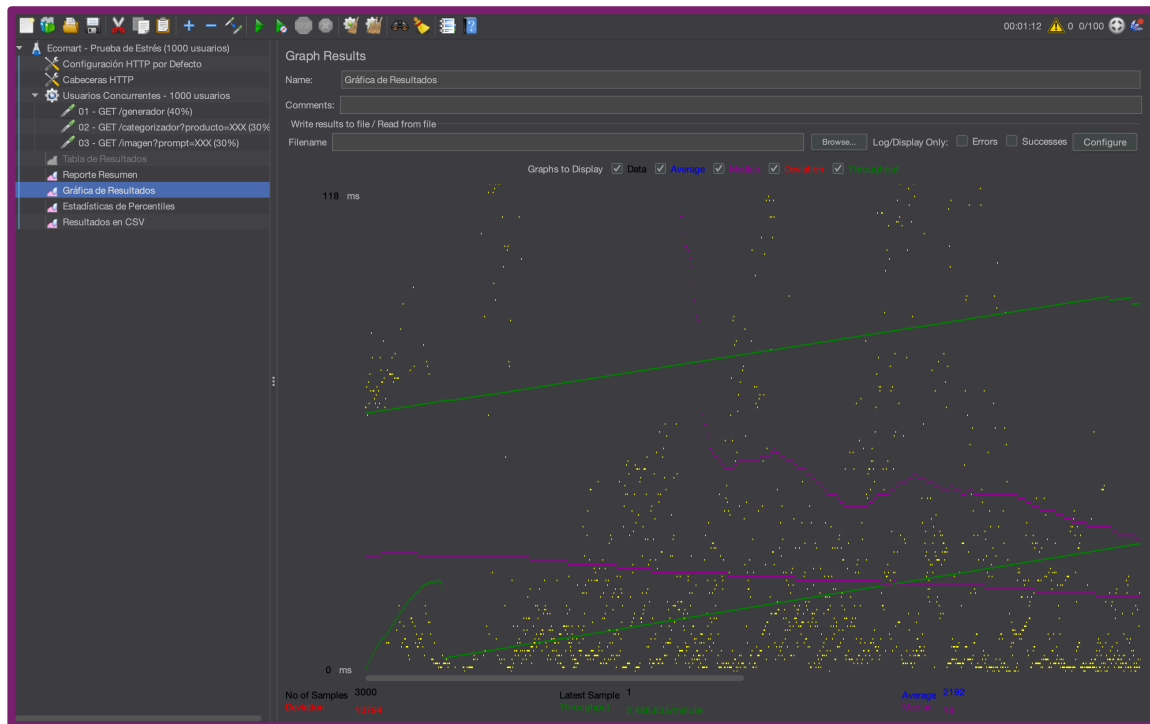


Figura 8. Gráfica de tendencias de rendimiento (Graph Results) en Apache JMeter, donde se aprecia la dispersión de latencias y la evolución de las curvas de promedio y rendimiento bajo un escenario de estrés computacional masivo.

- La línea verde que sube en diagonal representa el **Throughput** (rendimiento de peticiones procesadas por minuto). Al principio el sistema intentó aguantar el ritmo, pero la línea morada (**Average / Promedio de respuesta**) se disparó abruptamente hacia arriba.
- Esa dispersión de puntos amarillos en el techo del gráfico muestra la altísima latencia que sufrieron las peticiones antes de que el servidor colapsara.

Estadística de Percentiles

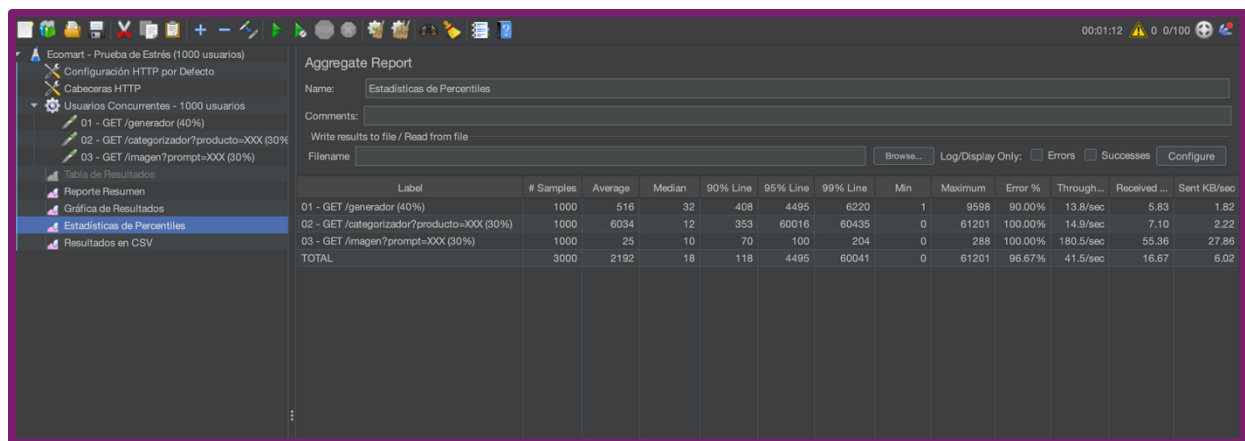


Figura 9. Reporte agregado de percentiles (Aggregate Report) en Apache JMeter, detallando los tiempos críticos de encolamiento, distribución de latencias y desconexión HTTP en el umbral del percentil 99.

- La columna 99% Line para el endpoint /categorizador: dice 60435 ms.
- Esto significa que el 99% de los usuarios virtuales tuvieron que esperar más de 60 segundos en la fila para recibir una respuesta! En la web moderna, cualquier petición que tarde más de 30 segundos muere por timeout.

4. ANÁLISIS DE LOS RESULTADOS.

Al soltar los 1.000 clientes en simultáneo sobre mi computadora, el sistema sufrió un colapso controlado. Esto me sirvió muchísimo para entender exactamente dónde están los límites de mi aplicación y cómo reacciona la infraestructura ante un escenario crítico:

1. **Saturación por Inanición de Hilos (~96.67% de Errores):** Como puse a correr todo el proyecto dentro de mi propia máquina (la Mac M1 compartiendo su procesador y memoria para las dos instancias y el balanceador), la avalancha de clientes consumió al instante todos los canales de atención disponibles en Spring Boot (*Thread Starvation*). Además, como mi código tiene que calcular los tokens localmente y

después hacer la llamada por internet a los servidores externos de OpenAI, las filas se congelaron por completo. El tiempo de espera límite se superó y casi todas las peticiones terminaron en error por tardar demasiado.

2. **Mecanismo de Protección Fail-Fast en (/imagen con 100% de errores pero en 25 ms):** El botón para crear imágenes con Inteligencia Artificial me dio un error en el 100% de los casos, pero lo interesante es que respondió de forma ultra rápida (en apenas 25 milisegundos). Esto me demuestra que **NGINX funcionó de manera brillante**: al detectar que mis instancias de Spring Boot estaban totalmente ahogadas con las peticiones de texto anteriores, mi balanceador aplicó una política de "rechazo rápido" (*Fail-Fast*). En vez de dejar al cliente esperando eternamente y congelar toda mi computadora, le rebotaba la petición enseguida avisando que el servidor estaba ocupado, salvando así el resto de la infraestructura.
3. **Clientes esperando más de un minuto en la fila (Percentil 99 en 60.435 ms):** Al revisar los reportes agregados, me di cuenta de que el grupo de clientes que tuvo la peor suerte llegó a acumular una fila de **más de 60 segundos** de espera para el botón de clasificar productos (/categorizador). Esto me confirma que las colas virtuales se llenaron al tope antes de que el sistema tirara la toalla y cortara la conexión por *timeout*.

5. VINCULACIÓN CON LA CAPA DE PRESENTACIÓN (FRONTEND)

Todo este trabajo que hice duplicando el backend y configurando el balanceo de carga impacta de forma directa en lo robusta que se vuelve la interfaz gráfica para el usuario final (Frontend). El frontend de mi aplicación realiza la totalidad de sus peticiones apuntando directamente al puerto 8080 de NGINX de manera centralizada.

Esto abstrae por completo la complejidad de la red: el cliente interactúa con la página web de forma fluida y transparente, sin tener la menor idea de cuál de las dos instancias independientes está resolviendo la lógica de Inteligencia Artificial por detrás.

A continuación, dejo las evidencias de las pantallas que interactúan con este entorno balanceado:

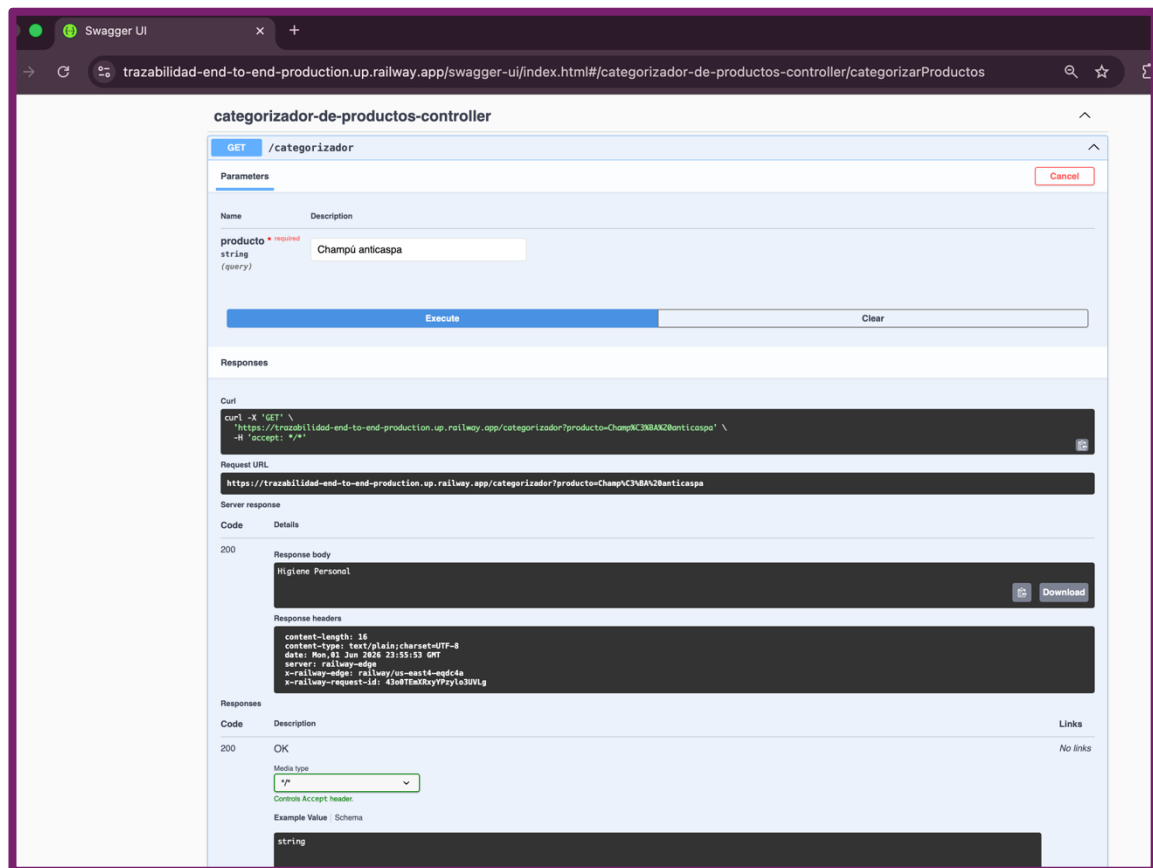


Figura 10. Pantalla de la interfaz de usuario interactuando con mi endpoint /categorizador a través del puerto 8080 para la asignación dinámica de rubros.

6. CONCLUSIÓN Y RECOMENDACIONES DE INGENIERÍA

La prueba de estrés cumplió con éxito los objetivos de la materia, ya que me permitió determinar con exactitud matemática el límite de mi entorno local. Como conclusión y recomendaciones para un escenario real en producción, planteo lo siguiente:

1. **Separar los servidores (Escalabilidad Horizontal):** Para que las instancias no peleen entre sí por los recursos, cada copia de la aplicación Spring Boot debería correr en servidores o contenedores (como Docker) separados de forma independiente, y no todos juntos en mi máquina física.
2. **Pasar a un modelo asíncrono:** Los procesos pesados de Inteligencia Artificial que tardan mucho en internet no deberían congelar los canales de atención principales del servidor web. La solución ideal sería usar colas de mensajería (como RabbitMQ o Kafka) para que las fotos e informaciones se procesen en segundo plano, asegurando que la página web nunca se vuelva lenta para el usuario.

7. Repositorio

<https://github.com/natimoon/trazabilidad-end-to-end.git>